Міністерство освіти і наук України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №8
Варіант 1
На тему
“Алгебраїчні типи даних “по-Python’івськи“

Виконав:
Студент групи ФЕП-21
Бардак Д. Т

Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- модель виразів **Expr = Num | Add | Mul**, «чистий» eval через match; **перевірка типів mypy**.
- **Union**, **Literal**, **Protocol**, структурний **pattern matching** (match/case) для розбору структур.

Теоретичні відомості

Алгебраїчні типи даних (ADT): «сума варіантів» (sum types). У Python моделюємо через **класи** і **Union**.
Literal - звужує значення до конкретних літералів (напр., Literal["+", "*"]).
Protocol - структурні інтерфейси (duck typing з перевіркою mypy).
match/case - структурний розбір: case Add(left=l, right=r).

Хід роботи

У ході виконання лабораторної роботи розроблено строго типізовану модель арифметичних виразів на базі алгебраїчних типів даних (ADT) засобами мови Python. Використовуючи декоратор @dataclass(frozen=True) для забезпечення повної незмінності стану, було створено набір вузлів абстрактного синтаксичного дерева (AST), які згодом об'єднано в загальний тип за допомогою Union та TypeAlias. Для точного обмеження допустимих арифметичних операцій застосовано тип Literal, а для гарантування структурної сумісності об'єктів — інтерфейси на основі typing.Protocol. Безпосереднє обчислення значень виразів, як і допоміжний алгоритм згортання констант (constant folding), реалізовано у вигляді чистих функцій, логіка яких базується винятково на структурному зіставленні зразків (match/case) без застосування громіздких перевірок isinstance. Додатково створено механізм текстового рендерингу AST та інтегровано захист від математичних помилок, зокрема винятки для ділення на нуль і переповнення. Коректність рекурсивних обчислень, структурного розбору та обробки граничних випадків успішно верифіковано автоматизованими тестами у фреймворку pytest, а архітектурну надійність проєкту підтверджено статичним аналізатором mypy в режимі строгих перевірок (--strict) разом з інструментами форматування black і ruff.



```
from __future__ import annotations

from eval_ import (
    MAX_ABS,
    DivisionByZeroError,
    EvalOverflow,
    eval_expr,
    eval_expr2,
    fold_constants,
)

__all__ = [
    "MAX_ABS",
    "DivisionByZeroError",
    "EvalOverflow",
    "eval_expr",
    "eval_expr2",
    "fold_constants",
]
```

Рис. 1 Файл evaluator.py

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Literal, Protocol, TypeAlias, Union

Op: TypeAlias = Literal["+", "-", "*", "/"]


class SupportsBinaryOp(Protocol):
    """Структурний контракт (Protocol) для бінарних вузлів AST."""

    @property
    def op(self) -> Op: ...

    @property
    def left(self) -> Expr: ...

    @property
    def right(self) -> Expr: ...


@dataclass(frozen=True)
class Num:
    value: int


@dataclass(frozen=True)
class Add:
    left: Expr
    right: Expr
    op: Literal["+"] = "+"


@dataclass(frozen=True)
class Sub:
    left: Expr
    right: Expr
    op: Literal["-"] = "-"


@dataclass(frozen=True)
class Mul:
    left: Expr
    right: Expr
    op: Literal["*"] = "*"


@dataclass(frozen=True)
class Div:
    left: Expr
    right: Expr
    op: Literal["/"] = "/"


@dataclass(frozen=True)
```

```python
class Bin:
    """Альтернативний бінарний вузол із явним використанням Literal Op."""

    op: Op
    left: Expr2
    right: Expr2


@dataclass(frozen=True)
class Num2:
    value: int


Expr: TypeAlias = Union[Num, Add, Sub, Mul, Div]  # noqa: UP007
Expr2: TypeAlias = Union[Num2, Bin]  # noqa: UP007
```

Рис. 2-4 Файл expr.py

```python
from __future__ import annotations

from eval_ import DivisionByZeroError, eval_expr, fold_constants
from expr import Add, Div, Mul, Num, Sub
from pretty import render


def main() -> None:
    print("Лабораторна №8: Алгебраїчні типи даних (Варіант 1)")

    # Вираз: ((10 - 2) * (3 + 0)) / 4
    ast = Div(
        Mul(Sub(Num(10), Num(2)), Add(Num(3), Num(0))),
        Num(4),
    )

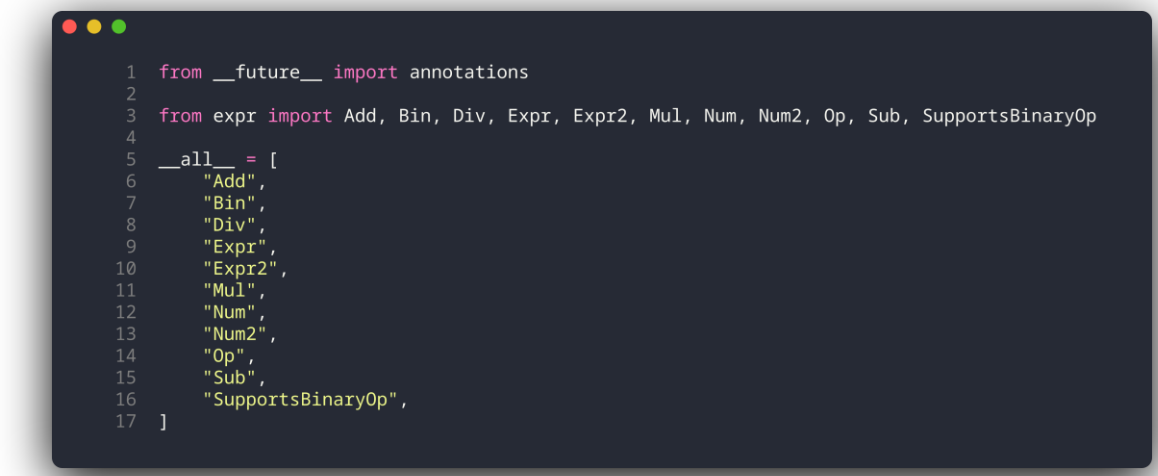
    print(f"Початковий вираз : {render(ast)}")
    print(f"Результат обчисл.: {eval_expr(ast)}")

    folded = fold_constants(ast)
    print(f"Після згортання  : {render(folded)} (тип вузла: {type(folded).__name__})")

    bad_ast = Div(Num(42), Sub(Num(5), Num(5)))
    print(f"Небезпечний вираз: {render(bad_ast)}")
    try:
        eval_expr(bad_ast)
    except DivisionByZeroError as err:
        print(f"Перехоплено помилку: {err}")


if __name__ == "__main__":
    main()
```

Рис. 5-6 Файл main.py

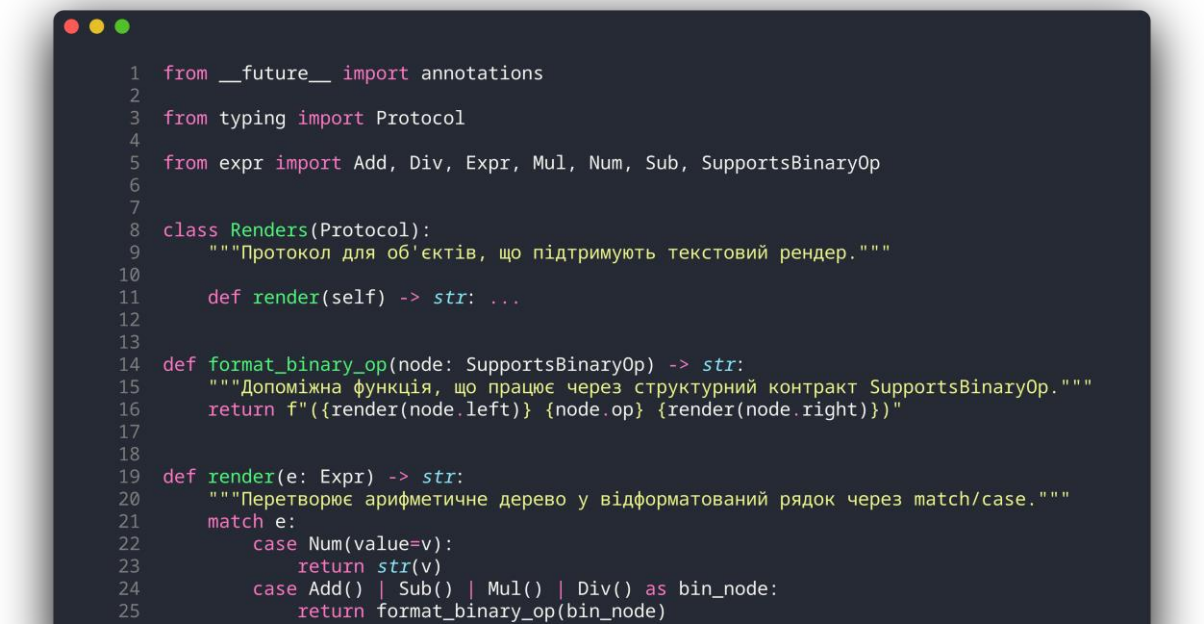


```python
from __future__ import annotations

from expr import Add, Bin, Div, Expr, Expr2, Mul, Num, Num2, Op, Sub, SupportsBinaryOp

__all__ = [
    "Add",
    "Bin",
    "Div",
    "Expr",
    "Expr2",
    "Mul",
    "Num",
    "Num2",
    "Op",
    "Sub",
    "SupportsBinaryOp",
]
```

Рис. 7 Файл model.py



```python
from __future__ import annotations

from typing import Protocol

from expr import Add, Div, Expr, Mul, Num, Sub, SupportsBinaryOp


class Renders(Protocol):
    """Протокол для об'єктів, що підтримують текстовий рендер."""

    def render(self) -> str: ...


def format_binary_op(node: SupportsBinaryOp) -> str:
    """Допоміжна функція, що працює через структурний контракт SupportsBinaryOp."""
    return f"({render(node.left)} {node.op} {render(node.right)})"


def render(e: Expr) -> str:
    """Перетворює арифметичне дерево у відформатований рядок через match/case."""
    match e:
        case Num(value=v):
            return str(v)
        case Add() | Sub() | Mul() | Div() as bin_node:
            return format_binary_op(bin_node)
```

Рис. 8 Файл pretty.py

```python
from __future__ import annotations

from typing import Final

from expr import Add, Bin, Div, Expr, Expr2, Mul, Num, Num2, Sub

MAX_ABS: Final[int] = 10_000


class EvalOverflow(ValueError):
    """Виняток при перевищенні допустимого діапазону значень."""


class DivisionByZeroError(ZeroDivisionError):
    """Виняток при діленні на нуль під час обчислення AST."""


def _check_range(x: int) -> int:
    if abs(x) > MAX_ABS:
        raise EvalOverflow(f"overflow: {x}")
    return int(x)


def eval_expr(e: Expr) -> int:
    """Чисте обчислення арифметичного дерева через структурний pattern matching."""
    match e:
        case Num(v):
            return _check_range(v)
        case Add(a, b):
            return _check_range(eval_expr(a) + eval_expr(b))
        case Sub(a, b):
            return _check_range(eval_expr(a) - eval_expr(b))
        case Mul(a, b):
            return _check_range(eval_expr(a) * eval_expr(b))
        case Div(a, b):
            denom = eval_expr(b)
            if denom == 0:
                raise DivisionByZeroError("Division by zero in expression")
            return _check_range(eval_expr(a) // denom)
        case _:
            raise RuntimeError(f"Невідомий вузол: {e}")


def fold_constants(e: Expr) -> Expr:
    """Згортання констант (constant folding) та базові алгебраїчні спрощення."""
    match e:
        case Num():
            return e
        case Add(a, b):
            fa, fb = fold_constants(a), fold_constants(b)
            match (fa, fb):
                case (Num(va), Num(vb)):
                    return Num(va + vb)
                case (Num(0), right_expr):
                    return right_expr
```



```
                case (left_expr, Num(0)):
                    return left_expr
                case _:
                    return Add(fa, fb)
        case Sub(a, b):
            fa, fb = fold_constants(a), fold_constants(b)
            match (fa, fb):
                case (Num(va), Num(vb)):
                    return Num(va - vb)
                case (left_expr, Num(0)):
                    return left_expr
                case _:
                    return Sub(fa, fb)
        case Mul(a, b):
            fa, fb = fold_constants(a), fold_constants(b)
            match (fa, fb):
                case (Num(va), Num(vb)):
                    return Num(va * vb)
                case (Num(0), _) | (_, Num(0)):
                    return Num(0)
                case (Num(1), right_expr):
                    return right_expr
                case (left_expr, Num(1)):
                    return left_expr
                case _:
                    return Mul(fa, fb)
        case Div(a, b):
            fa, fb = fold_constants(a), fold_constants(b)
            match (fa, fb):
                case (_, Num(0)):
                    return Div(fa, fb)
                case (Num(va), Num(vb)):
                    return Num(va // vb)
                case (left_expr, Num(1)):
                    return left_expr
                case _:
                    return Div(fa, fb)
        case _:
            return e


def eval_expr2(e: Expr2) -> int:
    """Обчислення альтернативної форми AST із Literal-операторами."""
    match e:
        case Num2(v):
            return _check_range(v)
        case Bin("+", a, b):
            return _check_range(eval_expr2(a) + eval_expr2(b))
        case Bin("-", a, b):
            return _check_range(eval_expr2(a) - eval_expr2(b))
        case Bin("*", a, b):
            return _check_range(eval_expr2(a) * eval_expr2(b))
        case Bin("/", a, b):
            denom = eval_expr2(b)
            if denom == 0:
                raise DivisionByZeroError("Division by zero in expression")
            return _check_range(eval_expr2(a) // denom)
        case _:
            raise RuntimeError(f"Невідомий вузол або оператор: {e}")
```

Рис. 9-12 Файл eval.py

```python
from __future__ import annotations

import os
import sys

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

import pytest

from eval_ import (
    DivisionByZeroError,
    EvalOverflow,
    eval_expr,
    eval_expr2,
    fold_constants,
)
from expr import Add, Bin, Div, Mul, Num, Num2, Sub
from pretty import render


def test_eval_basic() -> None:
    """Перевірка базових арифметичних операцій ADT."""
    assert eval_expr(Num(5)) == 5
    assert eval_expr(Add(Num(2), Num(3))) == 5
    assert eval_expr(Sub(Num(10), Num(4))) == 6
    assert eval_expr(Mul(Num(2), Num(3))) == 6
    assert eval_expr(Div(Num(8), Num(2))) == 4


def test_eval_nested() -> None:
    """Перевірка вкладених рекурсивних структур."""
    # ((2 + 3) * (10 - 2)) / 4 = (5 * 8) / 4 = 10
    ast = Div(Mul(Add(Num(2), Num(3)), Sub(Num(10), Num(2))), Num(4))
    assert eval_expr(ast) == 10


def test_render() -> None:
    """Перевірка форматування виразу у рядок."""
    ast = Div(Mul(Add(Num(2), Num(3)), Num(4)), Sub(Num(9), Num(7)))
    assert render(ast) == "(((2 + 3) * 4) / (9 - 7))"


def test_division_by_zero() -> None:
    """Перевірка захисту від ділення на нуль."""
    ast = Div(Num(10), Sub(Num(3), Num(3)))
    with pytest.raises(DivisionByZeroError):
        eval_expr(ast)


def test_overflow() -> None:
    """Перевірка винятку при переповненні MAX_ABS."""
    ast = Mul(Num(5000), Num(3))
    with pytest.raises(EvalOverflow):
        eval_expr(ast)


def test_constant_folding() -> None:
    """Перевірка згортання констант (constant folding)."""
    ast = Mul(Add(Num(2), Num(3)), Sub(Num(10), Num(6)))
    folded = fold_constants(ast)
    assert folded == Num(20)

    div_zero_ast = Div(Add(Num(1), Num(2)), Num(0))
    assert fold_constants(div_zero_ast) == Div(Num(3), Num(0))


def test_eval_expr2_literal() -> None:
    """Перевірка альтернативного AST із Literal-операторами."""
    ast = Bin(
        "/", Bin("*", Bin("+", Num2(2), Num2(3)), Num2(4)), Bin("-", Num2(6), Num2(4))
    )
    assert eval_expr2(ast) == 10
```

Рис. 13-15 Файл test_lab07.py

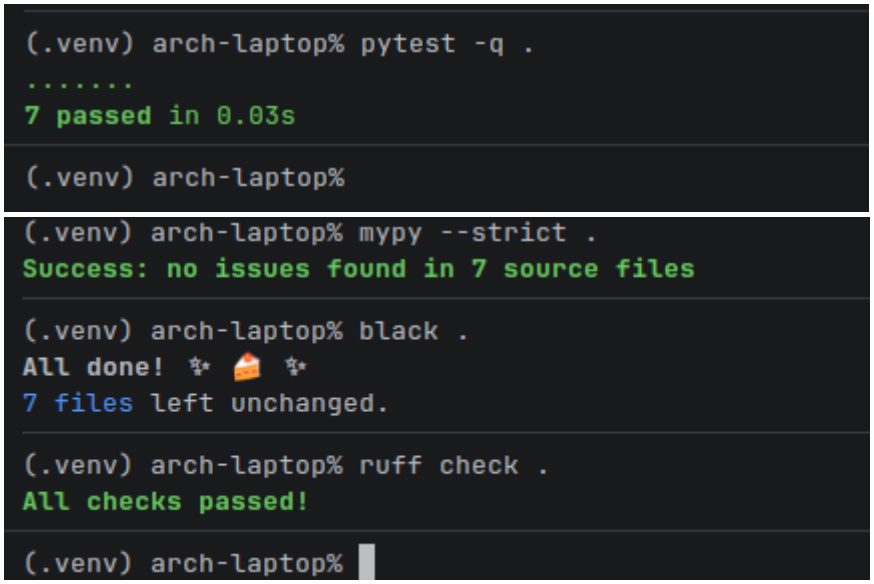


Рис.16-17 Успішно виконані тести

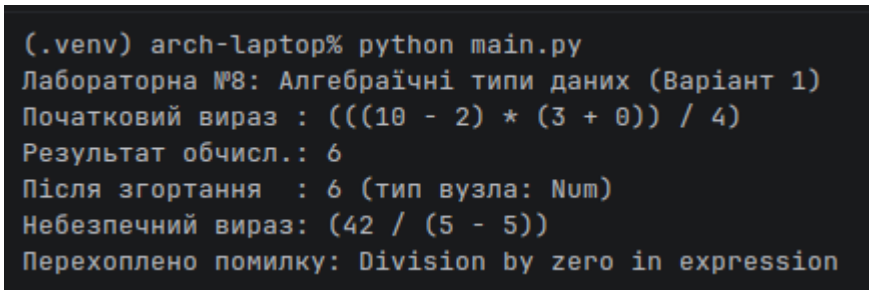


Рис. 18 Запуск програми

Висновок:

Під час лабораторної роботи було практично засвоєно парадигму роботи з алгебраїчними типами даних (ADT) "по-Python'івськи", використовуючи сучасні можливості стандарту мови (починаючи з Python 3.10). Доведено, що комбінація незмінних dataclass, об'єднань Union/TypeAlias та структурного зіставлення зразків (match/case) дозволяє писати чистий, декларативний код для обробки дерев та складних структур без використання методів всередині класів і громіздких перевірок isinstance. Впровадження Literal та Protocol забезпечило надійну типізацію, дозволяючи статичному аналізатору mypy в режимі --strict виявляти помилки розбору, невідповідності сигнатур та пропущені гілки логіки ще до запуску програми. Такий підхід робить архітектуру гнучкою, безпечною та ідеально пристосованою для побудови інтерпретаторів, компіляторів, аналізаторів граматики або складних станів бізнес-логіки.